

Programmation impérative en OCaml

Contents

I) Principes de base : référence vs valeur, mutable vs immutable	1
I.1) Valeurs	1
I.2) Références	1
I.3) Instruction vs expression	2
I.4) Boucles en OCaml	2
I.5) Fonctions anonymes et d'ordre supérieur	3

I) Principes de base : référence vs valeur, mutable vs immutable

Ces principes sont valables autant en Python que en OCaml.

I.1) Valeurs

On va voir un ordinateur comme un *processeur* accompagné d'une *mémoire*.

La mémoire, c'est genre une grande armoire, où sont rangées toutes les valeurs de vos variables.

Quand le processeur a besoin de faire un calcul sur une valeur, il la sort de l'armoire, fait ce qu'il a à faire, et la remet.

```
a = 3 + 4
c = a
```

Mais le processeur par défaut il est pas très organisé ! Donc il ne retiens pas où sont rangés les objets. Donc si on change de "contexte", il oublie tout ! Par exemple, en entrant dans une fonction ...

```
def plop(a):
    a = 9
```

```
a = 3
plop(a)
print(a)
```

[lien notebook](#)

Il se souviens que il a un 3 sur lui en entrant, mais il a complètement oublié où allait le 3 original ! Et donc il est incapable de le modifier.

En OCaml, on interdit au processeur d'écrire dans les variables directement : comme ça, pas d'ambiguïté.

I.2) Références

Maintenant, parfois on aimerait bien modifier des variables même dans des fonctions... Et donc il y a des mécanismes pour ça : les références.

Le principe, c'est qu'au lieu de donner la valeur directement, on donne sa localisation dans la mémoire. Comme ça, même si le processeur la prend pour faire des calculs avec, il saura où la ranger ensuite !

Des objets fonctionnant sur ce principe, vous en connaissez : les listes et les dictionnaires en python, les tableaux et les tables de hashage en OCaml.

Comme c'est des objets très gros, le processeur peut pas prendre toute la valeur de l'objet d'un coup, donc il retiens juste où sont stockés les éléments !

En python, les références sont implicites. C'est un peu confus, mais globalement, il n'y a que les listes et les dictionnaire qu'on peut modifier depuis une autre fonction. En particulier, les tuples ne sont pas passés par référence !

En OCaml, on peut faire des références *explicites*, c'est à dire qu'on dit à OCaml "ceci est une référence, laisse moi la modifier stp" et il est d'accord. [lien notebook](#)

On peut :

- créer une référence en ajoutant juste `ref` devant une valeur
- modifier la valeur dans une référence avec `:=` (et non simplement `=` !)
- accéder à la valeur dans une référence avec `!`

Globalement, il suffit de retenir ces deux lignes :

```
let a = ref 0;;  
a := !a + 1;;
```

Au passage, on a deux fonctions `incr` et `decr` qui permettent d'ajouter et d'enlever 1 à une référence d'entier.

Ainsi, `a := !a + 1` devient `incr a`.

On peut faire une référence de n'importe quoi ! Par exemple, une référence de liste, d'arbre, etc.

I.3) Instruction vs expression

En informatique, on distingue souvent les *expressions* des *instructions*. En python, la distinction est assez clair : une instruction, c'est une ligne entière. Une expression, c'est un truc qui a une valeur.

```
a = 1 + max(2, 3)
```

Ici, on a une instruction, et `1 + max(2, 3)` est une expression.

Une instruction peut être simplement composé d'une expression : par exemple, un appel de fonction.

En OCaml, c'est un petit peu plus compliqué : en vérité, on a uniquement des expressions, mais on appelle *instruction* les expressions qui renvoient `()` c'est à dire le type `unit` (en gros, rien).

Par exemple :

- `tableau.(i) <- 3` mettre une valeur dans un tableau
- `print_string "bonjour"` les fonctions d'impression
- `a := 8` les assignations de références
- et toute les fonctions qui renvoient `unit`

La particulier des instructions, c'est qu'on peut mettre un `;` après, ce qui permet d'enchaîner sur d'autres instructions. Tout à la fin de la suite d'instruction, on peut mettre une expression, qui sera "renvoyée" en quelque sorte.

Par exemple :

```
let b = (a := 3; 3);;
```

On a une instruction `a := 3` à l'intérieur d'une expression, et la valeur de cette expression est "3". Cette expression a des effets sur la variable `a`, elle la modifie. On dira que c'est une expression avec des *effets de bords*.

I.4) Boucles en OCaml

Tout ça pour arriver au point intéressant : oui, on peut faire des boucles en OCaml. Sans récursion.

[lien notebook](#)

I.5) Fonctions anonymes et d'ordre supérieur

On peut faire ce qu'on appelle des *fonction anonymes* en ocaml, c'est à dire des fonctions qui n'ont pas de nom. Par exemple :

```
fun x y -> x + y
```

L'avantage principale est lorsque'on a besoin de fonctions très simples pour les placer dans des *fonctions d'ordre supérieur*, c'est à dire des fonctions qui prennent en argument des fonctions.

Celles que vous pouvez utiliser sont principalement les fonctions `init`, `map` et `iter` sur les listes et array.

Par exemple :

```
List.init len f (* renvoie [f 0; f 1; f 2; ...; f (len - 1)]*)  
let l = List.init 10 (fun i -> 2 * i) (*renvoie [0; 2; 4; ...; 18]*)
```

```
List.iter (fun i -> print_int i) l (*print la liste l*)
```

```
List.map (fun i -> Array.make i 0) l (*renvoie une liste contenant des tableaux de  
taille 0, 2, 4, ..., 18, en suivant la liste l*)
```

Quasiment toutes les collections OCaml ont ces fonctions quasiment à l'identique !